

JP Seismic Tomography - Run Instructions

Step-by-step procedure for reproducing the numerical benchmark and generating its CSV and PNG outputs

1. Files required

The folder should contain the two Python programs and the three accompanying PDFs. The benchmark itself requires only the numerical Python program and the plotting program; the PDFs explain how the experiment was organised and what must be preserved.

File	Purpose
JP_Seismic_Tomography.py	Loads the JP Matrix Market file, constructs b , runs CG, PCG, LSQR and LSMR over the lambda sweep, and writes CSV results.
plot_JP.py	Reads the three benchmark CSV files and creates the PNG figures.
JP_Seismic_Introduction.pdf	Explains the scientific problem and the structure of the benchmark.
JP_Seismic_Run_Instructions.pdf	This document: explains the practical execution procedure.
JP_Seismic_Requirements_and_Restrictions.pdf	Lists the resource requirements and conditions that should not be changed when reproducing the work.

2. This is a CSF job, not a normal laptop calculation

The JP matrix is large enough that the benchmark should be treated as a high-performance computing workload. The dissertation execution was performed on the University of Manchester CSF using Slurm. The exact CSF login and authentication procedure is not reproduced here because it is University-specific and can change. Access to CSF and knowledge of the current login procedure are prerequisites for the reproduction.

3. Prepare a clean working directory

After logging in to CSF, create a directory for this benchmark. The exact location can be changed, but keeping the code, input data and generated outputs organised makes it much easier to diagnose problems.

```
mkdir -p ~/jp_seismic
cd ~/jp_seismic
```

Place JP_Seismic_Tomography.py and plot_JP.py in this directory. Do not rename the programs after copying them unless the commands and documentation are updated accordingly.

4. Check the Python environment

The numerical program uses NumPy, SciPy and pandas. The plotting program additionally uses Matplotlib. The dissertation work was carried out with a scientific Python environment on CSF. Current University module and environment names can change, so the environment should be checked rather than assuming that an old module command still exists.

```
python --version
python -c "import numpy, scipy, pandas, matplotlib; print('Scientific Python stack available')"
```

If the import test fails, use the current University-supported CSF procedure to select or create a compatible Python/Anaconda environment. Do not install packages into a shared system environment without following the University's rules.

5. Obtain the JP matrix

The numerical script can download the SuiteSparse JP archive itself when the `--download-data` option is supplied. If the matrix has already been downloaded, the program can instead search the directory supplied through `--data-dir`.

The default data directory is `suitesparse_data`. With the download option, the program retrieves the Harvard Seismology JP archive, extracts it and searches below that directory for `JP.mtx`.

```
python JP_Seismic_Tomography.py --download-data
```

For a controlled reproduction it is also reasonable to obtain the JP Matrix Market file using the current permitted SuiteSparse procedure and place it below `suitesparse_data`. The important point is that the matrix must be the JP dataset identified by the code, not a different seismic-tomography matrix.

6. Understand the dissertation lambda sweep before running

The archived dissertation results were produced using eight explicit regularisation values. To reproduce that sweep with the current script, pass them explicitly:

```
python JP_Seismic_Tomography.py --lambdas 1e-6,1e-5,1e-4,1e-3,1e-2,1e-1,1,10 --repeats 3
```

This is preferable to relying on the current adaptive defaults when the aim is to reproduce the dissertation's reported JP results. The current script's default adaptive mode uses multipliers of $1e-3$ through 1000 relative to the computed mean diagonal scale.

7. Submit the numerical calculation through Slurm

The dissertation benchmark was submitted as a Slurm workload rather than being run directly in an ordinary login shell. The exact partition and CPU availability are dependent on the current CSF configuration and the account being used. A generic job script should therefore be adapted to the current University instructions.

The dissertation execution used a 24-hour wall-time allocation. The practical memory requirement is discussed separately in the Requirements and Restrictions document. Resource requests should be checked against the current CSF policy before submission.

8. Example Slurm job structure

The following is a template showing the structure of the job rather than a claim about a permanent University partition or CPU limit. The values for account, partition and CPU count must be changed to match the current CSF rules.

```
#!/bin/bash
#SBATCH --job-name=jp_seismic
#SBATCH --time=24:00:00
#SBATCH --cpus-per-task=
#SBATCH --mem=

cd ~/jp_seismic

python JP_Seismic_Tomography.py \
--lambdas 1e-6,1e-5,1e-4,1e-3,1e-2,1e-1,1,10 \
--repeats 3
```

The current University documentation should be used to add the appropriate account, partition and any site-specific environment setup. The number of CPUs is user-dependent and should be checked before submitting the job. Increasing the requested CPU count does not automatically make the

numerical algorithms multithreaded in the same way as an explicit parallel implementation.

9. Submit and monitor the job

Save the job script, then submit it using the normal Slurm command available on CSF:

```
sbatch jp_seismic.slurm
```

Slurm will return a job identifier. That identifier is only for monitoring the particular submission and is not part of the benchmark specification.

```
squeue -u $USER  
sacct -j --format=JobID,State,Elapsed,MaxRSS
```

When the calculation finishes, check that the job completed rather than being cancelled, timed out or killed for exceeding memory. If a job fails, the Slurm accounting information and the program's standard output/error are the first places to look.

10. Check the numerical outputs

The current numerical program writes its results into the default output directory `real_data_jp_output`. The principal files are:

Output	What it contains
<code>JP_lambda_sweep_results.csv</code>	Detailed result for every solver, lambda value and repeated run.
<code>JP_lambda_sweep_summary.csv</code>	Grouped means, standard deviations and convergence information.
<code>JP_solver_runtime_comparison.csv</code>	Mean solver runtime by lambda and LSMR comparison ratios.

```
ls -lh real_data_jp_output
```

11. Generate the plots

Once the three CSV files exist, run the plotting program from the directory containing those files. The simplest form is:

```
cd real_data_jp_output  
python ../plot_JP.py
```

If the plotting script is run from the parent directory instead, supply the three input files explicitly and choose an output directory:

```
python plot_JP.py \  
--results real_data_jp_output/JP_lambda_sweep_results.csv \  
--summary real_data_jp_output/JP_lambda_sweep_summary.csv \  
--runtime real_data_jp_output/JP_solver_runtime_comparison.csv \  
--output real_data_jp_output/JP_plots
```

12. Expected plot files

The supplied plotting script creates ten PNG figures in `JP_plots`:

- `01_runtime_vs_lambda.png`
- `02_iterations_vs_lambda.png`
- `03_relative_residual_vs_lambda.png`
- `04_relative_normal_residual_vs_lambda.png`
- `05_objective_vs_lambda.png`

- 06_average_runtime_by_solver.png
- 07_average_iterations_by_solver.png
- 08_average_relative_residual_by_solver.png
- 09_lsmr_speedup_vs_competitors.png
- 10_convergence_success_by_solver.png

13. Copying results back for inspection

The exact transfer command depends on the user's local machine and the University's supported CSF transfer procedure. The important files to retrieve are the three CSV files and the generated PNG directory. The repository itself does not need to contain these generated files if the intention is to keep only source code and reproducibility documents.

14. Basic checks before accepting a reproduction

- The matrix is identified as JP from the Harvard Seismology group.
- The matrix dimensions are 87,616 by 67,320 with 13,734,559 non-zero entries.
- The right-hand side is constructed as $A @ \text{ones}$ rather than taken from an observational dataset.
- All four solvers are present: CG, PCG, LSQR and LSMR.
- The archived lambda sweep is reproduced explicitly when comparison with the dissertation is required.
- Three repeats are used for each solver and lambda value.
- The generated CSV files contain the expected solver and lambda combinations.
- The plotting script reads the generated CSVs rather than recomputing the numerical benchmark.